# P0260R14
# C++ Concurrent Queues

Published Proposal, 2025-01-12

**This version:**
http://wg21.link/P0260R14

**Authors:**
Lawrence Crowl
Chris Mysen
Detlef Vollmann
Gor Nishanov

**Audience:**
SG1, LEWG

**Project:**
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

**Source:**
gitlab.com/cppzs/papers/-/blob/main/source/P0260R14.bs

---

## Abstract

Concurrent queues are a fundamental structuring tool for concurrent programs. We propose concurrent queue concepts and a concrete implementation.

## Table of Contents

**References**

Informative References

## 1. Acknowledgments

Thanks to David Goldblatt, Dietmar Kühl and Jens Maurer for their help with the wording!

## 2. Revision History

This paper revises P0260R13 - 2024-12-10 as follows.

- updated wording based on feedback from Dietmar and Jens
- added example for non-blocking functions
- added discussion for std::expected
- added discussion for async sender behaviour

This paper revises P0260R12 - 2024-11-21 as follows.

- added `success` to `conqueue_errc`
- updated error handling
- added rationale for concepts
- updated API (error handling, return types, emplace), including rationale
- updated examples

This paper revises P0260R11 - 2024-10-12 as follows.

- Implemented feedback from LEWG and SG1 in St. Louis:
  - Added wording for allocator awareness
  - Added rationale for not providing single-ended interfaces
  - Added rationale for `bounded_queue` not being movable
  - Added rationale for `bounded_queue` not providing `emplace`
- Implemented feedback from SG1 in Wroclaw:
  - Fixed wording for sequential consistency

    - Fixed wording for `async_*`
    - Fixed ordering specification of `bounded_queue`
- Provided usage examples

This paper revises P0260R10 - 2024-06-26 as follows.

- Implement feedback from LEWG and SG1 in St. Louis:
- Make concept exposition only
- Require async operations to run on the scheduler of the receiver of the operation
- Call `set_error` for async operations if queue is closed
- Reintroduced accidentally removed data-race freeness from R9
- Require sequential constistent semantics for `bounded_queue`
- Remove `experimental` leftovers

P0260R10 revises P0260R9 - 2024-05-19 as follows.

- Implement feedback from SG1 in St. Louis:
- Split concept into three
- Require `try_*` to be lockfree (w/ error code `busy`)
- Drop `is_always_lock_free`
- Drop `capacity()`
- Removed discussion points forSG1
- Removed TS ship vehicle
- General cleanup in the design part

P0260R9 revises P0260R8 - 2024-03-08 as follows.

- Added more wording
- Fixed `capacity`
- Changed push/pop wording to use "strongly happens before"
- Added discussion points
- Removed paragraph about ctors/dtors returning on same thread
- `try_*` now never blocks (even for contention on internal synchronization)
- Wording for spurious failures of `try_*`

- The constructors for `bounded_queue` that pre-fill the queue have been removed
- Reference to P3282 added
- Moved discussion about `try_push(T &&, T &)` to historic contents.

P0260R8 revises P0260R7 - 2023-06-15 as follows.

- Restructured document
- Fix typos
- Implement LEWG feedback to derive conqueue_errc from system_error
- Implement LEWG feedback to add range constructor and go back to InputIterator
- size_t capacity() added
- Added TBB concurrent_bounded_queue as existing practice
- Moved discussion about pop() interface to separate paper
- reintegrated P1958 `buffer_queue`
- Renamed `buffer_queue` to `bounded_queue`
- Added proposed wording (incomplete)
- Updated TS questions

Older revision history was moved to after the proposed wording.

## 2.1. Discussion Points For This Revision

### 2.1.1. Discussion Points For SG1

- sequential consistency for push/pop

### 2.1.2. Discussion Points For LEWG

- Error handling: no exceptions, no `error_code` overloads
- Return type for `try_pop`
- Sender for `async_push`
- Should `conqueue_error` be dropped (and `conqueue_errc` renamed)?

# § 3. Introduction

Queues provide a mechanism for communicating data between components of a system.

The existing `deque` in the standard library is an inherently sequential data structure. Its reference-returning element access operations cannot synchronize access to those elements with other queue operations. So, concurrent pushes and pops on queues require a different interface to the queue structure.

Moreover, concurrency adds a new dimension for performance and semantics. Different queue implementation must trade off uncontended operation cost, contended operation cost, and element order guarantees. Some of these trade-offs will necessarily result in semantics weaker than a serial queue.

Concurrent queues come in a several different flavours, e.g.

- bounded vs. unbounded
- blocking vs. overwriting
- single-ended vs. multi-ended
- strict FIFO ordering vs. priority based ordering

The syntactic concepts proposed here should be valid for all of these flavours, while the concrete semantics might differ.

# § 4. Existing Practice

## § 4.1. Concept of a Bounded Queue

The basic concept of a bounded queue with potentially blocking push and pop operations is very old and widely used. It's generally provided as an operating system level facility, like other concurrency primitives.

POSIX 2001 has `mq` message queues (with priorities and timeout).

FreeRTOS, Mbed, vxWorks provide bounded queues.

## 4.2. Bounded and Unbounded Queues with C++ Interface

### 4.2.1. Literature

The first concurrent queue I've seen was in [Hughes97]. It was full of bugs and as such shows what will go wrong if C++ doesn't provide a standard queue. It's unbounded.

Anthony Williams provided a queue in *C++ Concurrency in Action*. It's unbounded.

### 4.2.2. Boost

Boost has a number of queues, as official library and as example uses of synchronization primitives.

*Boost Message Queue* only transfers bytes, not objects. It's bounded.

*Boost Lock-Free Queue* and *Boost Lock-Free SPSC Queue* have only non-blocking operations. *Boost Lock-Free Queue* is bounded or unbounded, *Boost Lock-Free SPSC Queue* is bounded.

*Boost Synchronized Queue* is an implementation of an early version of this proposal.

### 4.2.3. TBB

TBB has `concurrent_bounded_queue` (*TBB Bounded Queue*) and an unbounded version `concurrent_queue` *TBB Unbounded Queue*.

# 5. Examples and Implementation

## 5.1. Implementation

An implementation is available at github.com/GorNishanov/conqueue.

A free, open-source implementation of an earlier version of these interfaces is avaliable at the Google Concurrency Library project at github.com/alasdairmackintosh/google-concurrency-library. The original `buffer_queue` is in ..../blob/master/include/buffer_queue.h. The concrete `lock_free_buffer_queue` is in ..../blob/master/include/lock_free_buffer_queue.h. The corresponding implementation of the conceptual tools is in ..../blob/master/include/queue_base.h.

## § 5.2. Examples

Here we provide some examples how the API may be used.

The examples are available at https://gitlab.com/cppzs/conqueue/-/tree/wg21-demos/demo.

## § 5.3. Find Files with String

This example was presented in *Object-Oriented Multithreading Using C++*. The programm searches for files that contain a specific string. The idea is to have one thread to collect the file names from the filesystem and have several threads that read these files and searches them for the string.

Here we have some global definitions:

```
typedef std::bounded_queue<fs::path> FileList;

FileList *files;
std::string sWord;
```

Here's the function that searches for the files and pushes them into a queue:

```
void searchFiles(fs::path dir)
{
    if (fs::exists(dir) && fs::is_directory(dir))
    {
        fs::directory_iterator end;
        for (fs::directory_iterator i(dir); i != end; ++i)
        {
            if (!fs::is_directory(*i))
            {
                files->push(*i);                    // A
            }
        }
    }
    else
    {
        cerr << "no such directory" << endl;
    }
```

```
    files->close();                                 // B
}
```

At *A* the files are pushed into the queue and after all files are pushed at *B* the queue is closed.

Here's the function that picks a file and searches it for the string:

```
void searchWord()
{
    std::optional<fs::path> fname;

    while((fname = files->pop(ec)))                 // C
    {
        std::ifstream f(*fname);

        string line;
        while (getline(f, line))
        {
            if (line.find(sWord) != line.npos)
            {
                cout << "found in " << fname->string() << endl;
                break;
            }
        }
    }
}
```

Don't check for the queue being closed at *C*. Instead `pop`, and if the queue was closed *and no files a still in the queue* the returned optional will be empty.

And here's the `main` function to put everything together:

```
int main(int argc, char *argv[])
{
    sWord = argv[1];
    files = new FileList(100);                      // D

    thread a(searchFiles, argv[2]);
    thread b1(searchWord);
    thread b2(searchWord);
```

```
    a.join();
    b1.join();
    b2.join();

    delete files;

    return 0;
}
```

The queue is created (*D*) before the threads are started.

## § 5.4. Find Files with String (Async Coroutine Version)

> NOTE: this async version is only presented to demonstrate the use of the interface. Without an async `getline` and an async `directory_iterator` this async version doesn't make much sense.

For an async version (using coroutines) only very few things change:

`searchFiles` is now a coroutine returning `task<void>` that simply calls
`co_await files->async_push(*i);`
instead of
`files->push(*i);`

`searchWord` has a few more changes:

```
exec::task<void> searchWord(stdexec::run_loop *loop) // A
{
    std::optional<fs::path> fname;

    while ((fname = co_await (files->async_pop() | as_optional()))) // B
    {
        // ... same as before ...
    }

    loop->finish();                                  // C
}
```

At *B* we now `co_await async_pop`. We use `error_as_optional` (that we currently don't have) to get an optional again.

We now get a `run_loop` as parameter (*A*) on which we call `finish` (*C*) once we're done.

And `main` now sets up a `run_loop` and `async_scope` instead of starting threads:

```
int
main(int argc, char *argv[])
{
    stdexec::run_loop loop;
    exec::async_scope scope;

    sWord = argv[1];
    files = new FileList(100);

    scope.spawn(stdexec::on(loop.get_scheduler(), searchFiles(argv[2])));
    scope.spawn(stdexec::on(loop.get_scheduler(), searchWord(&loop)));

    loop.run();
    stdexec::sync_wait(scope.on_empty());

    delete files;

    return 0;
}
```

## § 5.5. Find Files with String (Async S/R Version)

This version looks pretty different.

The function that creates the sender to search for files looks like this:

```
template <class Case0, class Case1, class Case2>
exec::variant_sender<Case0, Case1, Case2>
branch(unsigned condition, Case0 sndr0, Case1 sndr1, Case2 sndr2)
{
    if (condition == 0) return std::move(sndr0);
    if (condition == 1) return std::move(sndr1);
    return std::move(sndr2);
```

```
}

unsigned iterSelect(fs::directory_iterator& i)
{
    unsigned ret = 2;
    if (i == fs::directory_iterator{})
    {
        ret = 0;
    }
    else
    {
        if (fs::is_directory(*i)) ret = 1;

        ++i;                                                    // A
    }

    return ret;
}

stdexec::sender auto
searchForFiles(std::bounded_queue<fs::path>& files,
               auto scheduler,
               fs::path dir)
{
    return stdexec::let_value(                                  // B
        stdexec::just(fs::directory_iterator(dir)),
        [&files, scheduler] (fs::directory_iterator& i)         // C
        {
            stdexec::sender auto nextFile = stdexec::let_value(  // D
                stdexec::just(),
                [&i, &files]
                {
                    fs::path file;
                    if (i != fs::directory_iterator{})
                    {
                        file = *i;                              // E
                    }

                    return branch(                              // F
                        iterSelect(i),

                        // Sndr0: close                         // G
                        stdexec::then(stdexec::just(),
```

```
                                    [&files]
                                    {
                                        std::cout << "closing\n";
                                        files.close();
                                        return true;
                                    }),

                        // Sndr1: skip directory          // H
                        stdexec::just(false),

                        // Sndr2: push file               // I
                        stdexec::then(files.async_push(file),
                                    [] { return false; }));
            });

            return exec::repeat_effect_until(           // J
                stdexec::starts_on(scheduler, nextFile));
        });
}
```

`let_value` at `B` ensures that the `i` at `C` is alive throughout its whole scope, i.e. for all iterations of `repeat_effect_until` at `J`. `let_value` at `D` ensures that `file` at `I` is alive for a single iteration.

`branch` at `F` is a helper that puts the correct sender into `variant_sender`. The correct sender is selected in `iterSelect` that also increments our iterator. We have three cases:

- The iterator is at the end, than we close the queue (`G`).
- The iterator points to a directory, then we skip the entry (`H`).
- Otherwise we push the entry into the queue (`I`).

The closing sender returns `true` to end the `repeat_effect_until`, the others return `false` to continue.

The function that creates the sender to search a single file for a string looks like this:

```
stdexec::sender auto searchWord(std::bounded_queue<fs::path>& files,
                                auto scheduler, std::string_view str) {
    stdexec::sender auto popAndProcess =
        files.async_pop()                                   // A
        | myexec::overload(
            []() -> bool { return true; },                  // B
            [str](const fs::path& fname) -> bool {          // C
```

```
                //std::cout << "searching " << fname.string() << std::endl;
                std::ifstream f(fname);
                std::string line;
                while (std::getline(f, line)) {
                    if (line.find(str) != line.npos) {
                        std::cout << "found in " << fname.string() << std::
                        break;
                    }
                }
                return false;
            });

    return exec::repeat_effect_until(
        stdexec::starts_on(scheduler, popAndProcess));
}
```

The sender created by `async_pop` sends two values:

- `void` if the queue is empty and closed;
- `fs::path` otherwise.

These two cases are covered by the `overload` algorithm. `B` covers the closed case and returns `true` to end the loop, `C` covers the `fs::path` case and processes the file.

## § 5.6. Logging

This is an example from an embedded system where no blocking is allowed.

Logging messages can be pushed into a queue from all kind of contexts, while a background task regularly pulls from the queue and sends messages to a serial interface.

This is the push function:

```
void enqueueLogMessage(std::string &&msg)
{
    if (logQ.try_push(std::move(msg)) != conqueue_errc::success)
    {
```

```
        ++lostLogMessages;
    }
}
```

This is the function that pulls the messages:

```
void outputLogMessage()
{
    static std::string bufferedString;

    unsigned busyCnt = 0;
    conqueue_errc ec;

    while (outputBuffer.add(bufferedString) > 0)
    {
        auto val = logQ.try_pop(ec);
        if (val)
        {
            bufferedString = std::move(*val);
            continue;
        }

        assert(ec != conqueue_errc::closed);
        if (ec == conqueue_errc::busy)
        {
            ++busyCnt;
            if (busyCnt > 2) break;
        }

        break;
    }
}
```

If the `try_pop()` interface would be changed to return `expected<T, conqueue_errc>` the only thing that would change is that the definition of `ec` would be moved to immediately before the `assert`:

```
// ...
conqueue_errc ec(val.error());
assert(ec != conqueue_errc::closed);
```

```
// ...
```

## § 6. Conceptual Interface

We provide basic queue operations, and then extend those operations to cover other important use cases.

Due to performance issues, the conceptual interface is split into three separate interfaces. The `basic_concurrent_queue` concept provides the interface `push` and `pop` and closing the queue. The `concurrent_queue` concept is based on `basic_concurrent_queue` and provides the additional interface `try_push` and `try_pop`. The `async_concurrent_queue` concept is also based on `basic_concurrent_queue` provides the additional interface `async_push` and `async_pop`.

The concrete queue `bounded_queue` models all these concepts.

There's no single queue implementation that can handle all use cases efficiently. Also not all use cases require both `try_*` and `async_*` interfaces, and and only providing one of them allows for more efficient implementations. Therefore it's expected that there will be many different implementations of these concepts with different trade-offs. Some of them might be standardized, most will be not. But the existence of the concepts allows for adapters (like single-ended interfaces to be used with different implementations. LEWG requested in St. Louis to make these concepts exposition-only, as this proposal doesn't include any adapters using these concepts.

### § 6.1. Error Class

By analogy with how `future` defines their errors, we introduce `conqueue_errc` enum and `conqueue_error`:

```
enum class conqueue_errc { success, empty, full, closed, busy };

const error_category& conqueue_category() noexcept;

class conqueue_error : public system_error;
```

As the queue functions don't throw their own errors anymore, probably only the enum will be kept.

## 6.2. Basic Operations

The essential solution to the problem of concurrent queuing is to shift to value-based operations, rather than reference-based operations.

The basic operations are:

```
bool queue::push(const T& x);
bool queue::push(T&& x);
template <typename... Args> bool emplace(Args &&... as);
```

Pushes `x` onto the queue via copy, move or argument based construction (and possibly blocks). It returns `true` on success, and `false` if the queue is closed.

```
std::optional<T> queue::pop();
```

Pops a value from the queue via move construction into the return value. If the queue is empty and closed, returns `std::nullopt`. If queue is empty and open, the operation blocks until an element is available.

The exploration of different version of error reporting was moved to a separate paper [P2921R0].

## 6.3. Non-Waiting Operations

Waiting on a full or empty queue can take a while, which has an opportunity cost. Avoiding that wait enables algorithms to do other work rather than wait for a push on a full queue, and to do other work rather than wait for a pop on an empty queue.

These operations will never block. If they would have to wait for internal synchronization, the queue status is `conqueue_errc::busy`.

```
conqueue_errc queue::try_push(const T& x);
conqueue_errc queue::try_push(T&& x);
template <typename... Args> conqueue_errc try_emplace(Args &&... as);
```

If no object could be placed into the queue, returns the respective status. Otherwise, pushes the value onto the queue via copy, move or argument based construction and returns `conqueue_errc::success`.

```
optional<T> queue::try_pop(conqueue_errc& ec);
```

If no object could be obtained from the queue, returns `nullopt` and sets ec to the queue status. Otherwise, pops the element from the queue via move construction into the optional and sets `ec` to `conqueue_errc::success`.

Alternative versions of the interface are

```
expected<T, conqueue_errc> queue::try_pop();
```

or even

```
expected<optional<T>, conqueue_errc> queue::try_pop();
```

[P2921R0] has the following example:

### 6.3.1. Drain the queue without blocking

```
conqueue_errc ec;
while (auto val = q.try_pop(ec))
   println("got {}", *val);
if (ec == conqueue_errc::closed)
  return;
// do something else.
```

`std::expected` based has unfortunate duplication since we want to know why the pop failed and we have to move `val` out of the loop condition.

```
auto val = q.try_pop();
while (val) {
  println("got {}", *val);
  val = q.try_pop();
}
```

```
if (val.error() == conqueue_errc::closed)
  return;
// do something else
```

With `expected<optional<T>, conqueue_errc>` it would look like this:

```
auto val = q.try_pop();
while (val && *val) {
  println("got {}", **val);
  val = q.try_pop();
}
if (val.error() == conqueue_errc::closed)
  return;
// do something else
```

In the LEWG discussion in St. Louis of [P2921R0] it was decided not to use expected:

POLL: LEWG would like to add a std::expected interface for concurrent queues.

| SF | F | N | A | SA |
|---|---|---|---|---|
| 0 | 2 | 5 | 3 | 2 |

However, in the meantime `try_pop` is the only function that still has an error code (`conqueue_errc`) parameter by reference. And given the logging example above it might make sense to revisit that decision.

## § 6.4. Asynchronous Operations

```
sender auto queue::async_push(T& x);
sender auto queue::async_push(T&& x);
template <typename... Args> sender auto async_emplace(Args &&... as);

sender auto queue::async_pop();
```

These operations return a sender that will push or pop the element. The operations support cancellation and if the receiver is currently waiting on a push or pop operation and no longer interested in performing the operation, it should be removed from any waiting queues, if any, and be completed with `std::execution::set_stopped`.

Sender based async interfaces have two main ways of usage: native S/R with receivers and coroutines where the receivers are provided by the coroutine library.

In Wroclaw only the coroutine usage was shown and LEWG voted there for an interface that favours coroutines. The sender returned by `async_pop()` would call `set_value(optional<T>)` which mirrors the synchronous `pop()`.

Afterwards it was pointed out that this interface is suboptimal for native receivers that can have multiple `set_value()` overloads.

So this revision proposes that `async_pop()` returns a sender that calls `set_value(void)` in the closed case and `set_value(T)` otherwise.

For the coroutine use this means that an additional adapter is required to combine these two into an optional:

```
while ((fname = co_await (files.async_pop() | as_optional))) {...}
```

The native sender/receiver usage could look like this:

```
files.async_pop()
| myexec::overload(
      []() -> bool { return true; },
      [str](const fs::path& fname) -> bool { ... return false; });
```

But a native sender/receiver usage could also provide an application specific receiver that now can handle the value case and the closed case completely separately.

By symmetry, `async_push` (and `async_emplace`) also call `set_value(void)` on a closed queue. For success they simply call `set_value(true_type)`.

An alternate option would be to actually handle the closed case as error. Now the coroutine users can decide to take the value from the sender directly, e.g.:

```
try {
    while (true) {
        fname = co_await files.async_pop();
        ifstream f(fname);
        // ...
    }
```

```
}
catch (conqueue_errc &e) {}
catch (system_error &e) { /* handle I/O error */ }
```

Or coroutine users could decide to still take it as optional:

```
while ((fname = co_await (files.async_pop() | error_as_optional())))
{
    ifstream f(*fname);
    // ...
}
```

## § 6.5. Closed Queues

Threads using a queue for communication need some mechanism to signal when the queue is no longer needed. The usual approach is add an additional out-of-band signal. However, this approach suffers from the flaw that threads waiting on either full or empty queues need to be woken up when the queue is no longer needed. To do that, you need access to the condition variables used for full/empty blocking, which considerably increases the complexity and fragility of the interface. It also leads to performance implications with additional mutexes or atomics. Rather than require an out-of-band signal, we chose to directly support such a signal in the queue itself, which considerably simplifies coding.

To achieve this signal, a thread may `close` a queue. Once closed, no new elements may be pushed onto the queue. Push operations on a closed queue will either return `conqueue_errc::closed` (when they have `ec` parameter) or throw `conqueue_error(conqueue_errc::closed)` (when they do not). Elements already on the queue may be popped off. When a queue is empty and closed, pop operations will either set `ec` to `conqueue_errc::closed` (when they have a `ec` parameter) or throw `conqueue_error(conqueue_errc::closed)` otherwise.

The additional operations are as follows:

```
void queue::close() noexcept;
```

Close the queue.

```
bool queue::is_closed() const noexcept;
```

Return true iff the queue is closed.

## 6.6. `emplace`

As queues have a `push` operation, it might be plausible to add an `emplace` operation as well. However, this might be misleading. `emplace` typically exist for performance reason for types that don't provide cheap move construction. So for containers where objects are meant to stay it makes sense to construct the objects in place. But queues aren't containers. Queues are mechanisms to push objects into them to be popped out again as soon as possible. So if the move of an object is expensive, it might be efficient to construct it inside a queue, but the pop would still be expensive. In such cases it might be more efficient to push a `unique_ptr` or similar through a queue instead of the actual values.

Providing an `emplace` operation might lead users to use a queue with object values and believe that's efficient.

However, `try_emplace` is definitely useful as no object is created at all if it's not inserted into the queue.

And `async_emplace` is useful as the construction of the object can be delayed to the point where `set_value(T)` is called which saves intermediate object construction.

And if `try_emplace` and `async_emplace` exist, for symmetry `emplace` should be provided as well, so this revision proposes them.

## 6.7. Element Type Requirements

The above operations require element types with copy/move constructors, and destructor. These operations may be trivial. The copy/move constructors operators may throw, but must leave the objects in a valid state for subsequent operations.

## 6.8. "Error" Handling

"One person's exception is another person's expected result."

It's extremely hard to define what results of a queue function actually constitutes an error. Instead of trying so, this proposal takes the approach that nothing is really an error.

Instead different function results due to different queue states are delivered by special return types.

`pop()` returns `optional<T>` (as decided in Wroclaw): empty `optional` for the `closed` state, and an optional containing a `T` otherwise.

`push()` returns `bool`: `false` for `closed` and `true` for successful enqueuing.

`try_pop` returns `expected<T, conqueue_errc>`: a `T` on success and the queue state otherwise.

`try_push` returns as `conqueue_errc` the queue state directly.

`async_pop()` returns a sender that calls `set_value(T)` if a value could be obtained from the queue and `set_value(void)` if the queue was closed.

`async_push()` returns a sender that calls `set_value(true_type)` if the value could be deposited and `set_value(void)` if the queue was closed.

Concurrent queues cannot completely hide the effect of exceptions thrown by the element type, in part because changes cannot be transparently undone when other threads are observing the queue.

Queues may rethrow exceptions from storage allocation or mutexes.

If the element type operations required do not throw exceptions, then only the exceptions above are rethrown. In practice, for a `bounded_queue` storage allocation only happens on construction and mutex `lock` and `unlock` generally don't throw.

When an element copy/move may throw, some queue operations have additional behavior.

- A push operation shall rethrow and the state of the queue is unaffected.
- A pop operation shall rethrow and the element is popped from the queue. The value popped is effectively lost. (Doing otherwise would likely clog the queue with a bad element.)

## § 6.9. Single-Ended Interfaces

In many applications one part only uses one end of a queue, while a different purt uses the other end. I.e. producer and consumer are logically separated. So it makes a lot of sense to provide interface type that only provide either the push or the pop interface.

But providing such separate interfaces requires additional overhead to ensure there arise no lifetime issues. So requiring such interfaces is not part of the concept.

And then it's assumed that a lot of different implementations of the concept will exist, with different tradeoffs. A separate type for the single-ended interfaces that works for all these implementation is

more efficent in terms of implementation. Such a type is not part of this proposal but should be a separate proposal. And as separate type it can easily be added later.

## § 7. Concrete Queues

In addition to the concepts, the standard needs at least one concrete queue. This will not provide the most efficient implementation but serves all synchronization use cases. For this reason it's bounded to allow for slowing down producers that are too fast.

This paper proposes a fixed-size `bounded_queue`. It meets all three concepts for queues. The constructor takes a parameter specifying the maximum number of elements in the queue and an optional allocator.

`bounded_queue` is only allowed to allocate in its constructor.

### § 7.1. Movability

The `bounded_queue` implementation will probably contain synchronization mechanisms like `std::mutes` and `std::condition_variable`. These are not movable, so `bounded_queue` isn't movable either.

### § 7.2. Memory Order Guarantees

`bounded_queue` provides sequentially consistent semantics.

The reasoning for this is that for a queue (as high-level data structure) provided by the IS usage safety should be more important than efficiency. Background on this is found in *Memory Model Issues for Concurrent Data Structures (P0387R1)* and *Concurrency Safety in C++ Data Structures (P0495)*.

The example from P0387R1 illustrates the problem clearly:

`q` and `log` are concurrent queues:

| Thread 1 | Thread 2 |
|---|---|
| `q.push(1);` | `log.push("pushed 2");` |
| `log.push("pushed 1");` | `q.push(2);` |

With only acquire/release this could result in `log` having `"pushed 1"` and `"pushed 2"`, while `q` has `2` and `1`.

`bounded_queue` will probably need a locking implementation anyway, which provides sequential consistency for free.

# 8. Proposed Wording

@@@!!!NOTE!!! This proposed wording is NOT up-to-date with the design above!

Suggested location of concurrent queues in the standard is [thread].

## 8.1. Header synopsis [version.syn]

#define __cpp_lib_conqueue // also in

## 8.2. Concurrent Queues [conqueues]

### 8.2.1. General [conqueues.general]

Concurrent queues provide a mechanism to transfer objects from one point in a program to another without producing data races.

### 8.2.2. Header `<conqueue>` synopsis [conqueues.syn]

```
namespace std {
  enum class conqueue_errc {
    success = implementation-defined,
    empty = implementation-defined,
    full = implementation-defined,
    closed = implementation-defined,
    busy = implementation-defined
  };

  template<> struct is_error_code_enum<conqueue_errc> : public true_type {
  const error_category& conqueue_category() noexcept;
  error_code make_error_code(conqueue_errc e) noexcept;
```

```
  error_condition make_error_condition(conqueue_errc e) noexcept;

  class conqueue_error;

  template <class T, class Allocator = std::allocator<T>>
  class bounded_queue;

  namespace pmr {
    template<class T>
    using bounded_queue = std::bounded_queue<T, polymorphic_allocator<T>>;
}
}
```

### 8.2.3. Error reporting [conqueue.error]

```
const error_category& conqueue_category() noexcept;
```

1. *Returns*: A reference to an object of a type derived from class `error_category`.
2. The object's `default_error_condition` and `equivalent` virtual functions shall behave as specified for the class `error_category`. The object's `name` virtual function shall return a pointer to the string "conqueue".

```
error_code make_error_code(conqueue_errc e) noexcept;
```

3. *Returns*: `error_code(static_cast<int>(e), conqueue_category())`.

```
error_condition make_error_condition(conqueue_errc e) noexcept;
```

4. *Returns*: `error_condition(static_cast<int>(e), conqueue_category())`.

### 8.2.4. Class `conqueue_error` [conqueue.errorclass]

```
namespace std {
  class conqueue_error : system_error {
  public:
```

```
    explicit conqueue_error(conqueue_errc e);

    const error_code& code() const noexcept;
    const char * what() const noexcept;

  private:
    error_code ec_;          // exposition only
  };
}
```

```
explicit conqueue_error(conqueue_errc e);
```

1. *Effects*: Initializes `ec_` with `make_error_code(e)`.

```
const error_code& code() const noexcept;
```

2. *Returns*: `ec_`.

```
const char * what() const noexcept;
```

3. *Returns*: An `NTBS` incorporating `code().message()`.

## 8.2.5. Exposition-only Concurrent Queue Concept [conqueue.concept]

### *8.2.5.1. Basic Concurrent Queue Concept [conqueue.concept.basic]*

1. The exposition-only concept *basic-concurrent-queue* defines the requirements for a basic queue type.

```
namespace std {
  template <class Q>
    concept basic-concurrent-queue = // exposition only
      move_constructible<remove_cvref_t<typename Q::value_type>> &&
      requires (Q q, const Q cq, typename Q::value_type &&t) {
        { cq.is_closed() } noexcept -> same_as<bool>;
        { q.close() } noexcept -> same_as<void>;
        { q.push(std::forward<typename Q::value_type>(t)) } -> same_as<
        { []<class... Args>(Q &q, Args... args) { bool b{q.emplace(std:
```

```
        { q.pop() } -> same_as<optional<typename Q::value_type>>;
      };
}
```

2. In the following description, `Q` denotes a type modeling the *basic-concurrent-queue* concept, `q` denotes an object of type `Q`, and `t` denotes an object convertible to `Q::value_type`.
3. The expression `q.is_closed()` has the following semantics:
    1. *Returns:* `true` if the queue is closed, `false` otherwise.
4. The expression `q.close()` has the following semantics:
    1. *Effects:* Closes the queue.
5. `q.push(std::forward<T>(t))` and `q.emplace(std::forward<Args>(args)...)` are *push operations* and `q.pop()` is a *pop operation*.
6. A push operation *deposits* an object into a queue. Hereby it is made available to be *extracted* from the queue A pop operation extracts and return an object from a queue. *push-deposited* is the event when the object is made available. *pop-claimed* is the event when a pop operation decides to extract the object.
7. If an object `t` is deposited by a push operation, the call of the constructor of this object inside the queue's storage strongly happens before the return of the pop operation that extracts `t`. A pop operation on a queue can only extract objects that were deposited into the same queue and each object can only be extracted once.
8. [*Note:* This doesn't specify when the *push-deposited* actually happens during the push operation, and likewise the *pop-claimed* during the pop operation. So it can happen that a pop operation already claimed a specific object but is blocked because the constructor of the object hasn't finished yet. -- end note]
9. Concepts that subsume *basic-concurrent-queue* may have additional push and pop operations.
10. Calls to operations (except constructor and destructor) on the same queue from different threads of execution do not introduce data races.
11. [*Note:* This concept doesn't specify whether a push may block for space available in the queue (bounded queue) or whether a push may allocate new space (unbounded queue). -- end note]
12. The expression `q.emplace(args...)` has the following semantics:
    1. *Effects:* If the queue is not closed, `t` is deposited into `q`. In this case the value in the queue storage is direct-non-list-initialized with `std::forward<Args>(args)...`.
    2. *Returns:*

- `true` if `t` was deposited into `q`; `false` otherwise.

3. *Throws:* Any exception thrown by the invoked constructor of `T`. A concrete queue may throw additional exceptions.

13. The expression `q.push(t)` is equivalent to `return q.emplace(std::forward<T>(t))`.

14. The expression `q.pop()` has the following semantics:

    1. *Effects:* Blocks the current thread until there is an object available in the queue or until the queue is closed. If the queue is closed return a default constructed `optional<T>`. Otherwise, if there is an object available in the queue, extract the object and return an `optional<T>` with a move-constructed value.

    2. *Returns:* as specified above.

    3. *Throws:* Any exception thrown by the invoked move constructor of `T`. A concrete queue may throw additional exceptions.

### § *8.2.5.2. Concurrent Queue Concept [conqueue.concept.concurrent]*

1. The exposition-only concept *concurrent-queue* defines the requirements for a concurrent queue type.

```
namespace std {
  template <class Q>
    concept concurrent-queue = // exposition only
      basic-concurrent-queue<Q> &&
      requires (Q q, typename Q::value_type &&t, conqueue_errc ec) {
        { q.try_push(std::forward<typename Q::value_type>(t)) } noexcep
        { []<class... Args>(Q &q, Args... args) { conqueue_errc e{q.try
        { q.try_pop(ec) } noexcept(see below) -> same_as<optional<typer
      };
}
```

2. `try_push`, `try_emplace` and `try_pop` are `noexcept` if the called constructor of `T` is `noexcept`.

3. In the following description, `Q` denotes a type modeling the *concurrent-queue* concept, `q` denotes an object of type `Q`, `t` denotes an object convertible to `Q::value_type` and `ec` denotes an object of type `conqueue_errc`.

4. `try_push` and `try_emplace` are push operations if they deposit an object into the queue and `try_pop` is a pop operation if it extracts an object from a queue.
5. In classes modelling this concept `try_push` or `try_emplace` may return `false` even if the queue has space for the element. Similarly `try_pop` may return `false` even if the queue has an element to be extracted. [*Note:* This spurious failure is normally uncommon. -- end note] An implementation should ensure that `try_push()` and `try_pop()` do not consistently spuriously fail.
6. The expression `q.try_emplace(t)` has the following semantics:
    1. *Effects:* If the queue is not closed, and space is available in the queue, `t` is deposited into `q`. In this case the value in the queue storage is direct-non-list-initialized with `std::forward<Args>(args)...`. The operation will not block.
    2. *Returns:*
        - `conqueue_errc::success` if `t` was deposited into `q`,
        - `closed` if the queue is closed,
        - `full` if the queue doesn't have space,
        - `busy` if the operation would block for internal synchronization.
7. The expression `q.try_push(t)` is equivalent to `return q.try_emplace(std::forward<T>(t))`.
8. The expression `q.try_pop(ec)` has the following semantics:
    1. *Effects:* If there is an object available in the queue and any internal synchronization wouldn't block, it will be extracted from the queue and an `optional<T>` with a move-constructed value returned, otherwise a default constructed `optional<T>` is returned. If a default constructed `optional<T>` is returned, `ec` is set to:
        - `closed` if the queue is closed,
        - `empty` if the queue doesn't have an object available,
        - `busy` if the operation would block for internal synchronization.

        The operation will not block.
    2. *Returns:* As specified above.

§ *8.2.5.3. Asynchronous Queue Concept [conqueue.concept.async]*

1. The exposition-only concept *async-concurrent-queue* defines the requirements for an asynchronous queue type.

```
namespace std {
  template <class Q>
    concept async-concurrent-queue =
      async-concurrent-queue<Q> &&
      requires (Q q, typename Q::value_type &&t) {
        { q.async_push(std::forward<typename Q::value_type>(t)) } noexc
        { q.async_emplace(std::forward<typename Q::value_type>(t)) } nc
        { []<class... Args>(Q &q, Args... args) { q.async_emplace(std::
        { q.async_pop() } noexcept;
      };
}
```

2. In the following description, `Q` denotes a type modeling the *async-concurrent-queue* concept, `q` denotes an object of type `Q` and `t` denotes an object convertible to `Q::value_type`.
3. `async_push` and `async_emplace` are push operations and `async_pop` is a pop operations.
4. The expression `q.async_emplace(args...)` has the following semantics:
    1. Let `w` be a sender object returned by the expression, `op` be an operation state obtained from connecting `w` to a receiver `r`.
    2. *Returns:* A sender object `w` that behaves as follows:
        1. After `op.start()` is called, a completion function of `r` is called when there s space in the queue or when the queue is closed.
        2. If there is space in the queue `t` will be deposited into the queue and `set_value(r)` will be called. The value in the queue storage is direct-non-list-initialized with `std::forward<Args>(args)...`.
        3. If the queue is closed `set_error(r, conqueue_errc::closed)` will be called.
        4. `set_value()` or `set_error()` will be called on the scheduler of `r`.
5. The expression `q.async_push(t)` is equivalent to `return q.async_emplace(std::forward<T>(t))`.
6. The expression `q.async_pop()` has the following semantics:

1. Let `w` be a sender object returned by the expression, `op` be an operation state obtained from connecting `w` to a receiver `r`.
2. *Returns:* A sender object `w` that behaves as follows:
    1. After `op.start()` is called, a completion function of `r` is called when there is an object available in the queue or when the queue is closed.
    2. If there is an object `t` available in the queue it will be extracted and `set_value(r, std::move(t))` will be called.
    3. If the queue is closed `set_error(r, conqueue_errc::closed)` will be called.
    4. `set_value()` or `set_error()` will be called on the scheduler of `r`.

### § 8.2.6. Class template `bounded_queue` [bounded.queue]

#### § *8.2.6.1. General [bounded.queue.general]*

1. A `bounded_queue` models *concurrent-queue* and *async-concurrent-queue* and can hold a fixed number of objects which is given at construction time.
2. 
```
template <class T,
  class Allocator = allocator<T>>
class bounded_queue
{
  bounded_queue(bounded_queue&&) = delete;

public:
  using value_type = T;
  using allocator_type = Allocator;

  // construct/destroy
  explicit bounded_queue(size_t max_elems, const Allocator& alloc = {})

  ~bounded_queue();

  // observers
  bool is_closed() const noexcept;

  allocator_type get_allocator() const;

  // modifiers
```

```
  void close() noexcept;

  bool push(const T& x);
  bool push(T&& x);
  template  bool emplace(Args &&... as);

  conqueue_errc try_push(const T& x);
  conqueue_errc try_push(T&& x);
  template  conqueue_errc try_emplace(Args &&... as);

  sender auto async_push(const T&);
  sender auto async_push(T&&);
  template  sender auto async_emplace(Args &&... as);

  optional<T> pop();
  optional<T> try_pop(conqueue_errc& ec);
  sender auto async_pop();

private:
  allocator_type alloc;      // exposition only
};
```

3. `T` shall be a type that meets the *Cpp17Destructible* requirements (Table [tab:cpp17.destructible]).
4. Template argument `Allocator` shall satisfy the *Cpp17Allocator* requirements ([allocator.requirements]). An instance of `Allocator` is maintained by the `bounded_queue` object during the lifetime of the object. The allocator instance is set at `bounded_queue` object creation time.
5. The values inside the storage of the queue deposited by push operations are constructed using `allocator_traits<Allocator>::construct(alloc, p, std::forward<T>(x))`, where `p` is the address of the element being constructed.

§ *8.2.6.2. Constructor [bounded.queue.ctor]*

```
explicit bounded_queue(size_t max_elems, const Allocator& alloc = Allocato
```

6. *Effects*: Constructs an object with no elements, but with storage for `max_elems`. The storage for the elements will be allocated using `alloc`.

7. *Remarks*: The operations of `bounded_queue` will not allocate any memory outside the constructor.

§ *8.2.6.3. Destructor [bounded.queue.dtor]*

```
~bounded_queue();
```

8. *Effects*: Destroys all objects in the queue and deallocates the storage for the elements using `alloc`.

§ *8.2.6.4. Allocator [bounded.queue.alloc]*

```
allocator_type get_allocator() const;
```

1. *Returns*: A copy of the `Allocator` that was passed to the object's constructor.

§ *8.2.6.5. Modifiers [bounded.queue.modifiers]*

1. The push and pop operations of `bounded_queue` provide sequentially consistent semantics.

```
void push(const T& x);
void push(T&& x);
template <class... Args> bool emplace(Args &&... as);
```

2. *Effects*: Blocks the current thread until there is space in the queue or until the queue is closed.
3. Let *Push1* and *Push2* be push operations and *Pop1* and *Pop2* be pop operations, where *Pop1* returns the value of the parameter given to *Push1* and *Pop2* returns the value of the parameter given to *Push2*. There is a total order consistent with the single total order `S` of `memory_order::seq_cst` operations [atomics.order]p4 of *push-deposited* and *pop-claimed* of *Push1*, *Push2*, *Pop1* and *Pop2*, and moreover if *push-deposited* of *Push1* is before *push-deposited* of *Push2* in that order, then *pop-claimed* of *Pop1* strongly happens before *pop-claimed* of *Pop2*.
4. It is unspecified whether the constructors and destructors of the objects in the internal storage of the queue run under a lock of the queue implementation.

5. [*Note:* This guarantees FIFO behaviour, but for two concurrent pushes the constructors cannot determine the order in which the values are enqueued, and the constructors can run concurrently as well. -- end note]
6. [*Note:* This does not guarantee that constructors or destructors may ever run concurrently. An implementation may decide that two pushes (or two pops) never run concurrently. -- end note]
7. [*Note:* A constructor or destructor can deadlock if it tries a push or pop on the same queue. -- end note]

```
T pop();
optional<T> pop();
optional<T> try_pop(conqueue_errc&);
sender auto async_pop();
```

8. *Effects*: The object in the queue is destroyed using `allocator_traits<Allocator>::destroy`.
9. *Remarks:* The constructor of the returned object and the destructor of the internal object run on the same thread of execution.

```
bool is_closed() const noexcept;
void close() noexcept;
conqueue_errc try_push(const T& x);
conqueue_errc try_push(T&& x);
template <class... Args> conqueue_errc try_emplace(Args &&... as);
sender auto async_push(const T&);
sender auto async_push(T&&);
template <class... Args> sender auto async_emplace(Args &&... as);
```

10. These functions behave as described in the respective concepts.

## § 9. Old Revision History

P0260R6 revises P0260R5 - 2023-01-15 as follows.

- Fixing typos.
- Added a scope for the target TS.
- Added questions to be answered by a TS.

- Added asynchronous interface

P0260R5 revises P0260R4 - 2020-01-12 as follows.

- Added more introductory material.
- Added response to feedback by LEWGI at Prague meeting 2020.
- Added section on existing practice.
- Replaced `value_pop` with `pop`.
- Replaced `is_lock_free` with `is_always_lockfree`.
- Removed `is_empty` and `is_full`.
- Added move-into parameter to `try_push(Element&&)`
- Added note that exception thrown by the queue operations themselves are derived from `std::exception`.
- Added a note that the wording is partly invalid.
- Moved more contents into the "Abandoned" part to avoid confusion.

P0260R4 revised P0260R3 - 2019-01-20 as follows.

- Remove the binding of `queue_op_status::success` to a value of zero.
- Correct stale use of the `Queue` template parameter in `shared_queue_front` to `Value`.
- Change the return type of `share_queue_ends` from a `pair` to a custom struct.
- Move the concrete queue proposal to a separate paper, [P1958].

P0260R3 revised P0260R2 - 2017-10-15 as follows.

- Convert `queue_wrapper` to a `function`-like interface. This conversion removes the `queue_base` class. Thanks to Zach Lane for the approach.
- Removed the requirement that element types have a default constructor. This removal implies that statically sized buffers cannot use an array implmentation and must grow a vector implementation to the maximum size.
- Added a discussion of checking for output iterator end in the wording.
- Fill in synopsis section.
- Remove stale discussion of `queue_owner`.
- Move all abandoned interface discussion to a new section.
- Update paper header to current practice.

P0260R2 revised P0260R1 - 2017-02-05 as follows.

- Emphasize that non-blocking operations were removed from the proposed changes.
- Correct syntax typos for noexcept and template alias.
- Remove `static` from `is_lock_free` for `generic_queue_back` and `generic_queue_front`.

P0260R1 revised P0260R0 - 2016-02-14 as follows.

- Remove pure virtuals from `queue_wrapper`.
- Correct `queue::pop` to `value_pop`.
- Remove nonblocking operations.
- Remove non-locking buffer queue concrete class.
- Tighten up push/pop wording on closed queues.
- Tighten up push/pop wording on synchronization.
- Add note about possible non-FIFO behavior.
- Define `buffer_queue` to be FIFO.
- Make wording consistent across attributes.
- Add a restriction on element special methods using the queue.
- Make `is_lock_free()` for only non-waiting functions.
- Make `is_lock_free()` static for non-indirect classes.
- Make `is_lock_free() noexcept`.
- Make `has_queue() noexcept`.
- Make destructors `noexcept`.
- Replace "throws nothing" with `noexcept`.
- Make the remarks about the usefulness of `is_empty()` and `is_full` into notes.
- Make the non-static member functions `is_...` and `has_...` functions `const`.

P0260R0 revised N3533 - 2013-03-12 as follows.

- Update links to source code.
- Add wording.
- Leave the name facility out of the wording.

- Leave the push-front facility out of the wording.
- Leave the reopen facility out of the wording.
- Leave the storage iterator facility out of the wording.

N3532 revised N3434 = 12-0124 - 2012-09-23 as follows.

- Add more exposition.
- Provide separate non-blocking operations.
- Add a section on the lock-free queues.
- Argue against push-back operations.
- Add a cautionary note on the usefulness of `is_closed()`.
- Expand the cautionary note on the usefulness of `is_empty()`. Add `is_full()`.
- Add a subsection on element type requirements.
- Add a subsection on exception handling.
- Clarify ordering constraints on the interface.
- Add a subsection on a lock-free concrete queue.
- Add a section on content iterators, distinct from the existing streaming iterators section.
- Swap front and back names, as requested.
- General expository cleanup.
- Add an "Revision History" section.

N3434 revised N3353 = 12-0043 - 2012-01-14 as follows.

- Change the inheritance-based interface to a pure conceptual interface.
- Put `try_...` operations into a separate subsection.
- Add a subsection on non-blocking operations.
- Add a subsection on push-back operations.
- Add a subsection on queue ordering.
- Merge the "Binary Interface" and "Managed Indirection" sections into a new "Conceptual Tools" section. Expand on the topics and their rationale.
- Add a subsection to "Conceptual Tools" that provides for type erasure.
- Remove the "Synopsis" section.

- Add an "Implementation" section.

# § 10. Historic Contents

**The Contents in this section is for historic reference only.**

## § 10.1. Abandoned Interfaces

### § 10.1.1. Re-opening a Queue

There are use cases for opening a queue that is closed. While we are not aware of an implementation in which the ability to reopen a queue would be a hardship, we also imagine that such an implementation could exist. Open should generally only be called if the queue is closed and empty, providing a clean synchronization point, though it is possible to call open on a non-empty queue. An open operation following a close operation is guaranteed to be visible after the close operation and the queue is guaranteed to be open upon completion of the open call. (But of course, another close call could occur immediately thereafter.)

```
void queue::open();
```

**Open the queue.**

Note that when `is_closed()` returns false, there is no assurance that any subsequent operation finds the queue closed because some other thread may close it concurrently.

If an open operation is not available, there is an assurance that once closed, a queue stays closed. So, unless the programmer takes care to ensure that all other threads will not close the queue, only a return value of true has any meaning.

Given these concerns with reopening queues, we do not propose wording to reopen a queue.

### § 10.1.2. Non-Blocking Operations

For cases when blocking for mutual exclusion is undesirable, one can consider non-blocking operations. The interface is the same as the try operations but is allowed to also return `queue_op_status::busy` in case the operation is unable to complete without blocking.

```
queue_op_status queue::nonblocking_push(const Element&);\queue_op_status
queue::nonblocking_push(Element&&);
```

**If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the `Element` onto the queue. Return `queue_op_status::success`.**

```
queue_op_status queue::nonblocking_pop(Element&);
```

**If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is empty, return `queue_op_status::empty`. Otherwise, pop the `Element` from the queue. The element will be moved out of the queue in preference to being copied. Return `queue_op_status::success`.**

These operations will neither wait nor block. However, they may do nothing.

The non-blocking operations highlight a terminology problem. In terms of synchronization effects, `nonwaiting_push` on queues is equivalent to `try_lock` on mutexes. And so one could conclude that the existing `try_push` should be renamed `nonwaiting_push` and `nonblocking_push` should be renamed `try_push`. However, at least Thread Building Blocks uses the existing terminology. Perhaps better is to not use `try_push` and instead use `nonwaiting_push` and `nonblocking_push`.

**In November 2016, the Concurrency Study Group chose to defer non-blocking operations. Hence, the proposed wording does not include these functions. In addition, as these functions were the only ones that returned `busy`, that enumeration is also not included.**

### § 10.1.3. Push Front Operations

Occasionally, one may wish to return a popped item to the queue. We can provide for this with `push_front` operations.

```
void queue::push_front(const Element&);\void queue::push_front(Element&&);
```

**Push the `Element` onto the back of the queue, i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.**

```
queue_op_status queue::try_push_front(const Element&);\queue_op_status
queue::try_push_front(Element&&);
```

**If the queue was full, return `queue_op_status::full`. Otherwise, push the `Element` onto the front of the queue, i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.**

```
queue_op_status queue::nonblocking_push_front(const Element&);\
queue_op_status queue::nonblocking_push_front(Element&&);
```

**If the operation would block, return `queue_op_status::busy`. Otherwise, if the queue is full, return `queue_op_status::full`. Otherwise, push the `Element` onto the front queue. i.e. in at the end of the queue that is normally popped. Return `queue_op_status::success`.**

This feature was requested at the Spring 2012 meeting. However, we do not think the feature works.

- The name `push_front` is inconsistent with existing \"push back\" nomenclature.
- The effects of `push_front` are only distinguishable from a regular push when there is a strong ordering of elements. Highly concurrent queues will likely have no strong ordering.
- The `push_front` call may fail due to full queues, closed queues, etc. In which case the operation will suffer contention, and may succeed only after interposing push and pop operations. The consequence is that the original push order is not preserved in the final pop order. So, `push_front` cannot be directly used as an \'undo\'.
- The operation implies an ability to reverse internal changes at the front of the queue. This ability implies a loss efficiency in some implementations.

In short, we do not think that in a concurrent environment `push_front` provides sufficient semantic value to justify its cost. Consequently, the proposed wording does not provide this feature.

### § 10.1.4. Queue Names

It is sometimes desirable for queues to be able to identify themselves. This feature is particularly helpful for run-time diagnotics, particularly when \'ends\' become dynamically passed around between threads. See Managed Indirection.

```
const char* queue::name();
```

**Return the name string provided as a parameter to queue construction.**

There is some debate on this facility, but we see no way to effectively replicate the facility. However, in recognition of that debate, the wording does not provide the name facility.

### § 10.1.5. Lock-Free Buffer Queue

We provide a concrete concurrent queue in the form of a fixed-size `lock_free_buffer_queue`. It meets the `NonWaitingConcurrentQueue` concept. The queue is still under development, so details may change.

**In November 2016, the Concurrency Study Group chose to defer lock-free queues. Hence, the proposed wording does not include a concrete lock-free queue.**

### 10.1.6. Storage Iterators

In addition to iterators that stream data into and out of a queue, we could provide an iterator over the storage contents of a queue. Such and iterator, even when implementable, would mostly likely be valid only when the queue is otherwise quiecent. We believe such an iterator would be most useful for debugging, which may well require knowledge of the concrete class. Therefore, we do not propose wording for this feature.

### 10.1.7. Empty and Full Queues

It is sometimes desirable to know if a queue is empty.

```
bool queue::is_empty() const noexcept;
```

**Return true iff the queue is empty.**

This operation is useful only during intervals when the queue is known to not be subject to pushes and pops from other threads. Its primary use case is assertions on the state of the queue at the end if its lifetime, or when the system is in quiescent state (where there no outstanding pushes).

We can imagine occasional use for knowing when a queue is full, for instance in system performance polling. The motivation is significantly weaker though.

```
bool queue::is_full() const noexcept;
```

**Return true iff the queue is full.**

Not all queues will have a full state, and these would always return false.

### 10.1.8. Queue Ordering

The conceptual queue interface makes minimal guarantees.

- The queue is not empty if there is an element that has been pushed but not popped.
- A push operation *synchronizes with* the pop operation that obtains that element.
- A close operation *synchronizes with* an operation that observes that the queue is closed.
- There is a sequentially consistent order of operations.

In particular, the conceptual interface does not guarantee that the sequentially consistent order of element pushes matches the sequentially consistent order of pops. Concrete queues could specify more

specific ordering guarantees.

### 10.1.9. Lock-Free Implementations

Lock-free queues will have some trouble waiting for the queue to be non-empty or non-full. Therefore, we propose two closely-related concepts. A full concurrent queue concept as described above, and a non-waiting concurrent queue concept that has all the operations except `push`, `wait_push`, `value_pop` and `wait_pop`. That is, it has only non-waiting operations (presumably emulated with busy wait) and non-blocking operations, but no waiting operations. We propose naming these `WaitingConcurrentQueue` and `NonWaitingConcurrentQueue`, respectively.

> NOTE: Adopting this conceptual split requires splitting some of the facilities defined later.

For generic code it\'s sometimes important to know if a concurrent queue has a lock free implementation.

```
constexpr static bool queue::is_always_lock_free() noexcept;
```

**Return true iff the has a lock-free implementation of the non-waiting operations.**

## 10.2. Abandoned Additional Conceptual Tools

There are a number of tools that support use of the conceptual interface. These tools are not part of the queue interface, but provide restricted views or adapters on top of the queue useful in implementing concurrent algorithms.

### 10.2.1. Fronts and Backs

Restricting an interface to one side of a queue is a valuable code structuring tool. This restriction is accomplished with the classes `generic_queue_front` and `generic_queue_back` parameterized on the concrete queue implementation. These act as pointers with access to only the front or the back of a queue. The front of the queue is where elements are popped. The back of the queue is where elements are pushed.

```
void send( int number, generic_queue_back<buffer_queue<int>> arv );
```

These fronts and backs are also able to provide `begin` and `end` operations that unambiguously stream data into or out of a queue.

### § 10.2.2. Streaming Iterators

In order to enable the use of existing algorithms streaming through concurrent queues, they need to support iterators. Output iterators will push to a queue and input iterators will pop from a queue. Stronger forms of iterators are in general not possible with concurrent queues.

Iterators implicitly require waiting for the advance, so iterators are only supportable with the `WaitingConcurrentQueue` concept.

```
void iterate(
    generic_queue_back<buffer_queue<int>>::iterator bitr,
    generic_queue_back<buffer_queue<int>>::iterator bend,
    generic_queue_front<buffer_queue<int>>::iterator fitr,
    generic_queue_front<buffer_queue<int>>::iterator fend,
    int (*compute)( int ) )
{
    while ( fitr != fend && bitr != bend )
        *bitr++ = compute(*fitr++);
}
```

Note that contrary to existing iterator algorithms, we check both iterators for reaching their end, as either may be closed at any time.

Note that with suitable renaming, the existing standard front insert and back insert iterators could work as is. However, there is nothing like a pop iterator adapter.

### § 10.2.3. Binary Interfaces

The standard library is template based, but it is often desirable to have a binary interface that shields client from the concrete implementations. For example, `std::function` is a binary interface to callable object (of a given signature). We achieve this capability in queues with type erasure.

We provide a `queue_base` class template parameterized by the value type. Its operations are virtual. This class provides the essential independence from the queue representation.

We also provide `queue_front` and `queue_back` class templates parameterized by the value types. These are essentially `generic_queue_front<queue_base<Value>>` and `generic_queue_front<queue_base<Value>>`, respectively.

To obtain a pointer to `queue_base` from an non-virtual concurrent queue, construct an instance the `queue_wrapper` class template, which is parameterized on the queue and derived from `queue_base`. Upcasting a pointer to the `queue_wrapper` instance to a `queue_base` instance thus erases the concrete queue type.

```
extern void seq_fill( int count, queue_back<int> b );

buffer_queue<int> body( 10 /*elements*/, /*named*/ "body" );
queue_wrapper<buffer_queue<int>> wrap( body );
seq_fill( 10, wrap.back() );
```

### 10.2.4. Managed Indirection

Long running servers may have the need to reconfigure the relationship between queues and threads. The ability to pass \'ends\' of queues between threads with automatic memory management eases programming.

To this end, we provide `shared_queue_front` and `shared_queue_back` template classes. These act as reference-counted versions of the `queue_front` and `queue_back` template classes.

The `share_queue_ends(Args ... args)` template function will provide a pair of `shared_queue_front` and `shared_queue_back` to a dynamically allocated `queue_object` instance containing an instance of the specified implementation queue. When the last of these fronts and backs are deleted, the queue itself will be deleted. Also, when the last of the fronts or the last of the backs is deleted, the queue will be closed.

```
auto x = share_queue_ends<buffer_queue<int>>( 10, "shared" );
shared_queue_back<int> b(x.back);
shared_queue_front<int> f(x.front);
f.push(3);
assert(3 == b.value_pop());
```

### 10.3. `try_push(T&&, T&)`

**REVISITED in Varna**

The following version was introduced in response to LEWG-I concerns about loosing the element if an rvalue cannot be stored in the queue.

`queue_op_status queue::try_push(T&&, T&);`

However, SG1 reaffirmed the APIs above with the following rationale:

It seems that it is possible not to loose the element in both versions:

```
T x = get_something();
if (q.try_push(std::move(x))) ...
```

With two parameter version:

```
T x;
if (q.try_push(get_something(), x)) ...
```

Ergonomically they are roughly identical. API is slightly simpler with one argument version, therefore, we reverted to original one argument version.

## References

### Informative References


**[BoostLFQ]**
*Boost Lock-Free Queue*. URL: https://www.boost.org/doc/libs/1_85_0/doc/html/boost/lockfree/queue.html

**[BoostLFSPSCQ]**
*Boost Lock-Free SPSC Queue*. URL: https://www.boost.org/doc/libs/1_85_0/doc/html/boost/lockfree/spsc_queue.html

**[BoostMQ]**
*Boost Message Queue*. URL: https://www.boost.org/doc/libs/1_85_0/doc/html/boost/interprocess/message_queue_t.html

**[BoostSQ]**

*Boost Synchronized Queue*. URL: https://www.boost.org/doc/libs/1_85_0/doc/html/thread/sds.html

**[Hughes97]**

Cameron Hughes; Tracey Hughes. *Object-Oriented Multithreading Using C++*.

**[P0387R1]**

*Memory Model Issues for Concurrent Data Structures (P0387R1)*. URL: https://wg21.link/P0387R1

**[P0495]**

*Concurrency Safety in C++ Data Structures (P0495)*. URL: https://wg21.link/P0495

**[P1958]**

*C++ Concurrent Buffer Queue (P1958)*. URL: https://wg21.link/P1958

**[P2921R0]**

Gor Nishanov, Detlef Vollmann. *Exploring std::expected based API alternatives for buffer_queue*. 5 July 2023. URL: https://wg21.link/p2921r0

**[TBBQB]**

*TBB Bounded Queue*. URL: https://spec.oneapi.io/versions/latest/elements/oneTBB/source/containers/concurrent_bounded_queue_cls.html

**[TBBQUB]**

*TBB Unbounded Queue*. URL: https://spec.oneapi.io/versions/latest/elements/oneTBB/source/containers/concurrent_queue_cls.html

**[Williams17]**

Anthony Williams. *C++ Concurrency in Action*.